SECURITY ASSESSMENT

Chitransh Mathur
CBS-0403

02/11/2025

Section 1:

Integrating SDLC (20)

Transitioning to Secure SDLC

Requirements Analysis

1. Conduct user interviews to gather functional requirements.
2. Write a requirements document for task management features.
3. Define cryptographic requirements (e.g threshold signatures, hardware-based key storage).
4. Perform threat modeling and risk assessment for cryptocurrency transactions and user wallets.
5. Identify regulatory and compliance requirements for crypto platforms.

Design

1. Create a high-level architecture diagram for the application.
2. Design the database schema for tasks.
3. Conduct Data Flow Diagram (DFD), Entity Relationship Diagram (ER) and identify trust boundaries.
4. Create a secure rollback plan with version integrity validation.
5. Conduct a cryptographic design review.
6. Implement data lineage design for sensitive crypto asset movements (provenance tracking).

Transitioning to Secure SDLC

Development

1. Code the user interface using HTML and CSS.
2. Implement interactive elements using JavaScript.
3. Set up a Flask application to handle API requests.
4. Implement CRUD operations for tasks.
5. Integrate Runtime Application Self-Protection (RASP) for anomaly detection at runtime.
6. Enforce memory-safe coding patterns and pointer sanitization checks especially in backend services.
7. Add secure serialization/deserialization policies to prevent object injection.

Testing

1. Write and execute functional test cases.
2. Conduct browser compatibility testing.
3. Execute TLS interception tests to verify certificate pinning and HSTS enforcement.
4. Include logic flaw testing, simulate unauthorized fund withdrawals, wallet reuse.
5. Conduct secure build pipeline integrity testing, e.g verify Git commit signing, checksum mismatches.

Transitioning to Secure SDLC

Deployment

1. Deploy the application to Heroku.
2. Perform smoke testing on the deployed application.
3. Deploy using immutable infrastructure, disallow SSH/RDP access entirely.
4. Configure signed and verified container registries with reproducible builds.
5. Introduce continuous security posture monitoring through policy-as-code tools.

Maintenance

1. Monitor application logs and fix reported issues.
2. Gather user feedback for future feature additions.
3. Implement real-time anomaly detection (like IDS) using ML models trained on transaction behaviors.
4. Periodic chaos-based incident response drills to measure team resilience.
5. Deploy deception mechanisms (honeypots/honey-tokens) to catch malicious insider or APT activity.

Advocating for Secure SDLC

1. Early Integration of Security
In Secure SDLC, security checks and threat modeling begin right from the requirement and design phases, instead of being delayed until testing. This early focus helps identify weaknesses such as insecure API flows or flawed crypto key handling before code is written, saving time, effort, and preventing costly rework later in development.
2. Continuous Feedback
Secure SDLC follows an iterative and adaptive model, where feedback from testing, monitoring, and real-world use continuously improves the product. For a dynamic platform like CryptoV4ult, where new cyber threats and compliance changes emerge regularly, this approach ensures the system evolves securely with each release cycle.
3. Stronger Incident Response
A Secure SDLC establishes clear incident response procedures, logging mechanisms, and recovery plans as part of maintenance. If a breach or outage occurs, teams can respond swiftly, minimize damage, and recover services without affecting user trust, ensuring the platform remains reliable and credible even during crises.

Advocating for Secure SDLC

4. Enhanced Software Quality and Reliability
Security-focused coding standards, dependency checks, and automated testing not only protect data but also improve the overall robustness of the software. With better code quality comes fewer production bugs, improved transaction accuracy, and a smoother user experience — essential in financial systems handling millions of transactions.
5. Secure-by-Design Architecture
Security isn’t added at the end it is architected into the system from the start, through data encryption, access control layers, input validation, and secure APIs. For CryptoV4ult, this means all services from wallet management to blockchain communication, are inherently resilient to attacks, minimizing risk of data leakage or unauthorized access.

Section 2:

Vulnerabilities and Remediation (35)

Vulnerabilities and remediation

1. Brute Force and Enumeration Attacks
Description
Some login systems reveal whether a username or password is incorrect through distinct error messages (e.g., “Invalid username” vs “Incorrect password”), allowing attackers to enumerate valid accounts. Additionally, unrestricted login attempts can allow brute-force attacks.
Risk
Attackers can systematically discover valid usernames and then attempt password guessing or automated attacks, eventually breaching user accounts. In the crypto domain, even one compromised admin account can be catastrophic.
Remediation
• Use generic error messages like “Invalid username or password”. • Implement account lockouts, progressive delays, and IP-based throttling. • Monitor authentication logs for suspicious patterns (same IP multiple accounts). • Integrate bot detection and anomaly scoring to block automated login attempts.

Vulnerabilities and remediation

2. Session Hijacking and Poor Session Management
Description
Weak session handling such as predictable session IDs, insecure cookies, or sessions that do not expire, can allow attackers to hijack valid user sessions and impersonate legitimate users.
Risk
An attacker who steals a session token (e.g., via XSS, packet sniffing, or malware) can perform sensitive operations like transferring crypto or changing withdrawal settings without reauthentication.
Remediation
• Use secure, randomly generated session IDs (UUID v4 or cryptographically random tokens). • Set cookies with `Secure`, `HttpOnly`, and `SameSite=Strict` flags. • Enforce automatic session expiration and re-authentication for critical actions (like wallet withdrawals). • Implement session invalidation on logout or password change. • Deploy TLS everywhere to prevent session token interception.

Vulnerabilities and remediation

3. Missing Multi-Factor Authentication (MFA)
Description
Relying solely on passwords for authentication exposes accounts to phishing, brute-force, or credential reuse attacks. Without MFA or device fingerprinting, attackers can log in even with stolen credentials.
Risk
If a user's password is compromised, their entire wallet and funds could be at risk. CryptoV4ult could face massive losses and regulatory scrutiny for inadequate access control mechanisms.
Remediation
• Implement MFA (e.g., TOTP, hardware keys, biometric authentication). • Allow device-based trust (e.g., verify login from new devices via email confirmation). • Employ risk-based authentication — request extra verification for suspicious logins (new IP, country, or device). • Educate users about phishing-resistant methods like FIDO2 or passkeys.

Threat Matrix

Pathway (Vulnerability)	Impact Level	Likelihood Level
Brute Force and Enumeration Attacks	High	High
Session Hijacking and Poor Session Management	High	Medium
Missing Multi-Factor Authentication (MFA)	High	High

Impact / Likelihood	Low	Medium	High
High			Missing Multi-Factor Authentication (MFA) & Brute Force and Enumeration Attacks
Medium			Session Hijacking and Poor Session Management
Low			

Section 3:

Container Security (20)

Trivy scan screenshot

```
┌──(kali㉿kali)-[~]
└─$ hostname
Chitransh

vulnerables/cve-2014-6271 (debian 7.11)

Total: 49 (UNKNOWN: 5, LOW: 0, MEDIUM: 12, HIGH: 20, CRITICAL: 12)
```

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
apache2 auth_digest	CVE-2018-1312	CRITICAL	fixed	2.2.22-13+deb7u12	2.2.22-13+deb7u13	httpd: Weak Digest auth nonce generation in mod_ https://avd.aquasec.com/nvd/cve-2018-1312
	CVE-2017-15710	HIGH				httpd: Out of bounds write in mod_authnz_ldap when using too small Accept-Language ... https://avd.aquasec.com/nvd/cve-2017-15710
	CVE-2018-1301	MEDIUM				httpd: Out of bounds access after failure in reading the HTTP request ... https://avd.aquasec.com/nvd/cve-2018-1301
apache2-mpm-worker auth_digest	CVE-2018-1312	CRITICAL				httpd: Weak Digest auth nonce generation in mod_ https://avd.aquasec.com/nvd/cve-2018-1312
	CVE-2017-15710	HIGH				httpd: Out of bounds write in mod_authnz_ldap when using too small Accept-Language ...

Report to Fix Container Issues

Vulnerability Name	Unpatched Software Version	Patched Software Version
CVE-2018-1312	2.2.22-13+deb7u12	2.2.22-13+deb7u13
CVE-2014-6271	4.2+dfsg-0.1	4.2+dfsg-0.1+deb7u1
CVE-2014-7169	4.2+dfsg-0.1	4.2+dfsg-0.1+deb7u3
CVE-2016-7543	4.2+dfsg-0.1	4.2+dfsg-0.1+deb7u4
CVE-2017-12613	1.4.1-3	1.4.1-3+deb7u1
CVE-2018-1126	1:3.3-3	1:3.3-3+deb7u1
CVE-2018-0739	1.0.1t-1+deb7u2	1.0.1t-1+deb7u3

Section 4:

API Security (15)

Vulnerabilities and remediation

1. Excessive Data Exposure
Description
APIs often send more information than necessary, leaving sensitive data visible in JSON responses. For instance, /api/user/profile might return the full user object (including email, phone, date of birth, KYC documents, wallet addresses, etc.) even though the vendor only needs a few fields for advertising segmentation.
Risk
• Attackers (or even legitimate vendors) can harvest unnecessary personal or financial data. • Increases attack surface for data misuse, identity theft, and targeted phishing. • Violates data minimization principles under privacy regulations.
Remediation
• Return only necessary fields — never expose full objects by default. • Apply field-level encryption or masking for sensitive values before exposure. • Introduce role-based serialization (e.g., an admin view returns more fields than a vendor view). • Conduct data mapping and privacy impact assessments (PIA) to identify and limit sensitive data flow.

Vulnerabilities and remediation

2. Broken Object Level Authorization (BOLA)
Description
BOLA occurs when an API exposes endpoints that allow users to access or modify data objects without properly validating ownership or authorization. For example, if the sales API lets a vendor query /user/1234, but fails to ensure that vendor A is only allowed to see *their assigned* customers, they could access other users' personal details.
Risk
• **Data exposure:** Unauthorized vendors could access or modify user profiles belonging to other partners. • **Privacy breach:** Personal and financial information leakage leads to GDPR, PCI DSS, or cryptocurrency KYC compliance violations. • **Reputation loss:** Users lose trust in CryptoV4ult's platform integrity and data governance.
Remediation
• Enforce per-user or per-tenant authorization before returning any object. • Use centralized access control logic (not client-side checks). • Include least privilege principles, vendors should access only the minimal data required for their function. • Conduct unit and integration tests that explicitly verify unauthorized access attempts fail.

Vulnerabilities and remediation

3. Improper Assets Management
Description
This happens when developers forget to properly track, secure, or retire old API endpoints such as deprecated test APIs, staging versions or debug interfaces left open in production. These endpoints often lack authentication, use outdated logic, or expose sensitive data structures that are supposed to be internal only.
Risk
• **Unauthorized access**: Attackers or vendors can discover and exploit unprotected or outdated endpoints. • **Compliance violations**: Insecure or undocumented APIs can lead to regulatory noncompliance (GDPR, PCI DSS, etc.). • **Attack surface expansion**: Every forgotten endpoint is an unmonitored doorway into the system, prime for reconnaissance and exploitation.
Remediation
• Maintain a complete API inventory, all endpoints, versions, owners, and purposes must be documented. • Enforce authentication and authorization on every API, even for internal or test endpoints. • Use an API gateway or service mesh to manage all traffic and ensure legacy endpoints are not exposed directly.